

DOMAIN DECOMPOSITION METHODS WITH PHYSICS-INFORMED NEURAL NETWORKS FOR ELLIPTIC EQUATIONS ON MANIFOLDS

YUFANG JIANG ^{*}, LIZHEN QIN [†], AND FENG WANG [‡]

Abstract. We propose two numerical domain decomposition methods (DDMs) for elliptic equations on compact Riemannian manifolds, based on physics-informed neural networks (PINNs). Our approach incorporates the DDM technique for manifolds with the advantages of neural networks in high-dimensional settings. The proposed methods are validated through numerical experiments on various manifolds, both with and without boundary, in dimensions ranging from 5 to 10.

AMS subject classifications. Primary 65N55; Secondary 58J05, 68T07.

Key words. Riemannian manifolds, elliptic problems, domain decomposition methods, physics-informed neural networks

1. Introduction. Elliptic equations on Riemannian manifolds are important in both analysis and geometry (see e.g. [3, 41, 67]). These equations appear in many areas, such as image processing, multifluid dynamics, micromagnetics, and theoretical physics (see e.g. [4, 5, 8, 18, 35, 36, 37, 39, 40, 57, 66]). A simple and important example is

$$(1.1) \quad -\Delta u + bu = f.$$

Here Δ is the Laplace-Beltrami operator, or Laplacian for brevity, defined on a d -dimensional Riemannian manifold M . Many manifolds of interest arise naturally as submanifolds of Euclidean spaces, where the “dimension” of a submanifold is understood as the topological dimension of the manifold, not that of the ambient Euclidean space. For instance, the unit sphere

$$S^{n-1} = \{x \in \mathbb{R}^n \mid \|x\| = 1\}$$

in $\mathbb{R}^n$ has dimension $n - 1$.

When the manifold M is a two-dimensional Riemannian submanifold in $\mathbb{R}^3$, i.e. a surface, the numerical methods to solve differential equations, particularly (1.1), on M have been extensively studied over many years (see e.g. [7, 22, 23, 56, 58]). A conventional and popular approach is to solve the equations by finite element methods (FEMs) based on a global grid or triangulation of M . Such a grid can be obtained by polyhedron approximation in $\mathbb{R}^3$. This approach has been highly developed and widely applied (see e.g. [9, 17, 24] for surveys and bibliographies).

However, as pointed out in [12] and [62], the above approach on surfaces faces substantial obstacles when extended to general high-dimensional manifolds, due to their great topological and geometrical complexity. Moreover, many important manifolds are even not defined as submanifolds of Euclidean spaces. A case in point is the complex projective space $\mathbb{CP}^k$ which is foundational to algebraic geometry. Such manifolds pose formidable challenges in numerical computation, particularly in the construction of global triangulations.

^{*}School of Mathematical Sciences, Nanjing Normal University, Nanjing Jiangsu, China (240902008@njnu.edu.cn).

[†]School of Mathematics, Nanjing University, Nanjing, Jiangsu, China (qinlz@nju.edu.cn).

[‡]School of Mathematical Sciences, Nanjing Normal University, Nanjing Jiangsu, China (fwang@njnu.edu.cn).

To circumvent this difficulty, Qin-Zhang-Zhang in [63] proposed an idea to solve elliptic equations on manifolds using FEMs without global triangulations. Since a d -dimensional manifold M has local coordinate charts by definition, M can be decomposed into finitely many subdomains that carry local coordinates. Consequently, an elliptic equation on each subdomain can be transformed to one on a domain in $\mathbb{R}^d$. Thus an elliptic problem on M can be solved by domain decomposition methods (DDMs) with subproblems posed on Euclidean domains, where grids are much easier to generate.

The idea of [63] was further developed by Cao-Qin in [12] and by Qin-Wang-Wang in [62]. Those works merge the philosophy in [63] and the seminal works of P. L. Lions. To solve elliptic differential equations in Euclidean domains, Lions proposed a sequential DDM in [47, Section I. 4] and a parallel DDM in [48, p. 66]. Both methods are overlapping DDMs formulated at continuous level, i.e. they are not numerical methods. These two DDMs were adapted and generalized to manifolds in [12] and [62], respectively. Furthermore, [12] and [62] also proposed DDMs based on FEMs on manifolds which are discrete analogues of the aforementioned continuous DDMs. Numerical experiments demonstrated that these numerical DDMs perform well on various 4-dimensional manifolds with and without boundary. The present work further extends these previous studies by incorporating neural network techniques.

In fact, as the dimension increases, FEMs face significant challenges. Due to the curse of dimensionality, for a fixed mesh size, the size of the stiffness matrix grows exponentially with respect to the dimension (cf. [13]). Though the numerical methods in [12] and [62] are theoretically applicable in any dimension given sufficient computing resources, they are only feasible in practice when the dimension is not too high.

To tackle problems on high-dimensional manifolds, in this paper, we combine the continuous DDMs in [12] and [62] with the method of physics-informed neural networks (PINNs). Following [12] and [62], we convert a global elliptic problem on a general d -dimensional manifold M into subproblems on a collection of domains in $\mathbb{R}^d$. Each subproblem is then solved using the PINNs method. In other words, we propose two overlapping DDMs, one sequential and one parallel, that employ PINNs at the numerical level for solving elliptic problems on manifolds. The PINNs method, popularized by [65], offers many advantages; we primarily exploit its ability to mitigate the curse of dimensionality. We test our methods on various manifolds, with and without boundary, in dimensions raging from 5 to 10. Numerical results indicate that these DDMs perform well.

Neural networks also have the merit of being mesh-free. Nonetheless, even though the difficulty of grid construction is eliminated, the framework of DDMs remains valuable for numerical computations on manifolds because coordinates are still indispensable for any numerical implementation. Since a general manifold M does not necessarily admit a global coordinate chart, it's difficult to construct a global neural network on a complicated M , especially when M is not defined as a submanifold of $\mathbb{R}^n$. A more practical strategy, following [63], is to cover M by a collection of subdomains M_i , each lying within a local coordinate chart. The global problem on M can then be solved numerically using these local coordinate systems.

DDMs have a very long history, beginning with the alternating method invented by H. A. Schwarz [68]. The seminal works [47, 48, 49] of P. L. Lions are natural and remarkable extensions of Schwarz's original idea. Since then, the field has grown significantly (see e.g. [20, 64, 70, 75]). In many later developments, DDM schemes, both overlapping and nonoverlapping, are used as preconditioners for globally discretized problems (see e.g. [1, 10, 11, 21, 77, 79]); within this con-

text, Lions’ sequential and parallel DDMs are related to the multiplicative and additive Schwarz methods, respectively. Neural networks have also been under active development for decades (see e.g. [6, 14, 16, 44, 46, 60]). In recent years, the field has seen rapid progress, and neural networks have emerged as a powerful and versatile numerical tool with broad impact across computational mathematics (see e.g. [2, 15, 25, 26, 27, 28, 31, 32, 33, 38, 50, 51, 54, 55, 59, 61, 65, 76, 78, 80]). For the numerical solution of differential equations (linear or nonlinear) on Euclidean domains, numerous studies have combined DDM frameworks with neural network approaches (see e.g. [19, 34, 43, 45, 52, 53, 69, 71, 72, 73]).

It is worth emphasizing that our work differs substantially from the earlier works on Euclidean-domain problems mentioned above. First, we apply DDMs to general high-dimensional manifolds. Particularly, we avoid constructing a global neural network on the target manifold. Instead of employing preconditioner techniques that rely on global networks, we follow more closely Lions’ original approach, which solves local subproblems exclusively. Furthermore, in our methods, data exchange between subdomains is carried out via coordinate transition functions, reflecting a fundamental feature of manifolds.

The outline of this paper is as follows. In Section 2, we introduce the model problem and recall the continuous-level DDMs proposed in [12] and [62]. In Section 3, we propose numerical DDMs based on PINNs which are numerical analogues of the continuous ones. Finally, some numerical results are reported in Section 4.

2. Theory on Continuous Problems. In this section, we shall first formulate an elliptic model problem of second order on manifolds. Then we recall the domain decomposition methods (DDMs) developed in [12] and [62] to solve the model problem (see Algorithms 2.1 and 2.2 below). These DDMs originated from the seminal works [47, 48] of P. L. Lions to solve differential equations in domains in Euclidean spaces. The iterative procedures also motivate us to propose numerical procedures based on physics-informed neural networks (PINNs) (Algorithms 3.1 and 3.2) in §3, which provide numerical approximations to the solution to the model problem.

2.1. Model Problem. Let M be a d -dimensional compact smooth manifold without or with boundary ∂M . Equip M with a Riemannian metric g . With respect to g , the Laplace operator Δ , also named the Laplace-Beltrami operator, is defined on M . Note that neither g nor Δ can be expressed by coordinates globally in general because M does not necessarily have a global coordinate chart. In a local chart with coordinates $(x_1, \dots, x_d)$, the Riemannian metric tensor g is expressed as

$$(2.1) \quad g = \sum_{\alpha, \beta=1}^d g_{\alpha\beta} dx_\alpha \otimes dx_\beta,$$

where the matrix $(g_{\alpha\beta})_{d \times d}$ is symmetric and positive definite. The Laplace operator Δ is expressed as

$$\Delta u = \frac{1}{\sqrt{G}} \sum_{\alpha=1}^d \frac{\partial}{\partial x_\alpha} \left(\sum_{\beta=1}^d g^{\alpha\beta} \sqrt{G} \frac{\partial u}{\partial x_\beta} \right),$$

where $G = \det((g_{\alpha\beta})_{d \times d})$ is the determinant of the matrix $(g_{\alpha\beta})_{d \times d}$ and $(g^{\alpha\beta})_{d \times d}$ is the inverse of $(g_{\alpha\beta})_{d \times d}$. It’s well-known that Δ is an elliptic differential operator of second order.

We consider the following model problem on M

$$(2.2) \quad \begin{cases} Lu = -\Delta u + bu = f, \\ u|_{\partial M} = \varphi, \end{cases}$$

where $b \geq 0$ is a constant. It suffices to solve (2.2) on each component of M . Therefore, without loss of generality, M is assumed to be connected. Note that, if $\partial M = \emptyset$, the boundary condition $u|_{\partial M} = \varphi$ is vacuously satisfied. However, to guarantee that (2.2) is well-posed, we need to further assume $b > 0$ if $\partial M = \emptyset$.

In a local chart, the operator L in (2.2) can be expressed via coordinates as

$$(2.3) \quad \begin{aligned} Lu &= -\frac{1}{\sqrt{G}} \sum_{\alpha=1}^d \frac{\partial}{\partial x_\alpha} \left(\sum_{\beta=1}^d g^{\alpha\beta} \sqrt{G} \frac{\partial u}{\partial x_\beta} \right) + bu \\ &= -\sum_{\alpha,\beta=1}^d g^{\alpha\beta} \frac{\partial^2 u}{\partial x_\alpha \partial x_\beta} - \frac{1}{\sqrt{G}} \sum_{\beta=1}^d \sum_{\alpha=1}^d \frac{\partial}{\partial x_\alpha} \left(g^{\alpha\beta} \sqrt{G} \right) \frac{\partial u}{\partial x_\beta} + bu. \end{aligned}$$

This is an elliptic operator with C^∞ coefficients and is in a quite general form since the Riemannian metric g can be arbitrary.

We assume that the problem (2.2) is well-posed, i.e. it has a unique solution u . The well-posedness is granted by the ellipticity of (2.2) provided that both f and φ have a certain mild regularity (cf. [29, Chapters 6 & 8] and [3, Chapter 3]).

2.2. Continuous Algorithms. We describe domain decomposition methods (DDMs) to solve (2.2). Suppose M is decomposed into m subdomains with $m > 1$, i.e. $M = \bigcup_{i=1}^m M_i$. Here M_i is a closed subdomain (submanifold with codimension 0) of M . Let $\gamma_i = \partial M_i \setminus \partial M$. We see $\overline{\gamma_i} \subseteq \partial M_i$, and $M_i \setminus \overline{\gamma_i}$ is the interior of M_i in the sense of point set topology. If $M_i \cap \partial M = \emptyset$, particularly $\partial M = \emptyset$, then $\overline{\gamma_i} = \gamma_i = \partial M_i$. We make the following assumption on decompositions.

ASSUMPTION 2.1. *Suppose $M = \bigcup_{i=1}^m (M_i \setminus \overline{\gamma_i})$, and each ∂M_i is Lipschitz.*

By Assumption 2.1, these $M_i \setminus \overline{\gamma_i}$ form an open cover of M . So our decomposition is overlapping. We firstly formulate a sequential DDM, i.e. Algorithm 2.1. It was originally proposed by P. L. Lions in [47, I. 4] to solve equations in Euclidean domains, in which the classical Schwarz Alternating Method is extended to an iterative procedure with many subdomains. Later, this method was generalized by [12, Algorithm 2.1] to solve equations on manifolds.

Lions described Algorithm 2.1 as “sequential” (see the sentence right before (26) in [47]), which means the subproblems on subdomains M_i have to be solved one by one sequentially. On the other hand, Lions later invented in [48, p. 66] another DDM to solve equations in Euclidean domains. The later one was claimed as “parallel” (see a few sentences before (37) in [48]). This parallel procedure was adapted and generalized to solve equations on manifolds in [62, Algorithm 2.1].

The adaption in [62] needs a partition of unity $\{\rho_i \mid 1 \leq i \leq m\}$ on M . In other words, ρ_i ’s are a family of nonnegative functions on M and $\sum_{i=1}^m \rho_i = 1$.

ASSUMPTION 2.2. *The partition of unity $\{\rho_i \mid 1 \leq i \leq m\}$ is subordinate to the decomposition in Assumption 2.1, i.e. $\text{supp} \rho_i \subset M_i \setminus \overline{\gamma_i}$, where $\text{supp} \rho_i$ is the support of ρ_i . Furthermore, each ρ_i is continuous.*

Let’s recall the parallel DDM in [62], i.e. Algorithm 2.2.

Algorithm 2.1 ([47, 12]) Sequential DDM to solve (2.2), under Assumption 2.1.

- 1: Choose an initial guess function u^0 on M with $u^0|_{\partial M} = \varphi$.
- 2: For each $n > 0$,
 define $u_0^n = u^{n-1}$.
 For $1 \leq i \leq m$,
 (assuming u_j^n has been obtained for all $j < i$.) find a function u_i^n on M such that

$$\begin{cases} Lu_i^n = f, & \text{in } M_i \setminus \partial M_i; \\ u_i^n = u_{i-1}^n, & \text{otherwise.} \end{cases}$$

Let $u^n = u_m^n$.

Algorithm 2.2 ([62]) Parallel DDM to solve (2.2), under Assumptions 2.1 and 2.2.

- 1: Choose an initial guess function u^0 on M with $u^0|_{\partial M} = \varphi$.
- 2: For each $n > 0$,
 for $1 \leq i \leq m$,
 find a function u_i^n on M_i such that

$$\begin{cases} Lu_i^n = f, & \text{in } M_i \setminus \partial M_i; \\ u_i^n = u^{n-1}, & \text{otherwise.} \end{cases}$$

Let $u^n = \sum_{i=1}^m \rho_i u_i^n$.

There are two obvious differences between the above two algorithms. First, unlike Algorithm 2.1, the parallel Algorithm 2.2 solves the subproblems on subdomains M_i independently and parallelly at each iterative step n . Second, each u_i^n in Algorithm 2.1 is globally defined on M , but each u_i^n in Algorithm 2.2 is only defined on M_i . Note that the definition of u^n in Algorithm 2.2 does make sense because $\text{supp} \rho_i \subset M_i$ for each i . (More precisely, the function $\rho_i u_i^n$ on M_i is extended as 0 outside of M_i .)

Algorithm 2.2 certainly follows [48, p. 66] in large. However, it is necessary to point out that, even in the special case where the manifold M is a Euclidean domain, Algorithm 2.2 is different from [48]. Actually, Algorithm 2.2 employs a partition of unity which was not utilized in [48]. (For a detailed description of this essential difference, see [62, p. A920].) Due to the difference, it has been proved in [62] that Algorithm 2.2 has a better convergence theory (see [62, Remark 2.5]).

If M is decomposed into two overlapping subdomains, i.e. $M = \bigcup_{i=1}^2 (M_i \setminus \overline{\gamma_i})$, Algorithms 2.1 and 2.2 are both reduced to the famous Schwarz Alternating Method which was invented by H. A. Schwarz ([68]) to solve differential equations in Euclidean domains. The following Algorithm 2.3 is the Schwarz Alternating Method on manifolds.

In Algorithm 2.3, one can extend u_1^n and u_2^n to be functions on M by defining $u_1^n|_{M \setminus M_1} = u_2^{n-1}$ and $u_2^n|_{M \setminus M_2} = u_1^n$. Therefore, Algorithm 2.3 is clearly a special case of Algorithm 2.1 since the alternating procedure is sequential in nature.

We point out that, Algorithm 2.2 also degenerates to Algorithm 2.3 for a two-subdomains decomposition. First of all, now a partition of unity is superfluous to the updating of boundary values. We necessarily have $\rho_i|_{\overline{\gamma_i}} \equiv 0$ and $\rho_i|_{\overline{\gamma_{3-i}}} \equiv 1$ for

Algorithm 2.3 Schwarz Alternating Method to solve (2.2), under Assumption 2.1 with $m = 2$.

- 1: Choose an initial guess functions u_2^0 on M_2 with $u_2^0|_{\partial M} = \varphi$.
- 2: For each $n > 0$,
 find a function u_1^n on M_1 such that

$$\begin{cases} Lu_1^n = f, & \text{in } M_1 \setminus \partial M_1; \\ u_1^n = u_2^{n-1}, & \text{on } \gamma_1; \\ u_1^n = \varphi, & \text{otherwise.} \end{cases}$$

Find a function u_2^n on M_2 such that

$$\begin{cases} Lu_2^n = f, & \text{in } M_2 \setminus \partial M_2; \\ u_2^n = u_1^n, & \text{on } \gamma_2; \\ u_2^n = \varphi, & \text{otherwise.} \end{cases}$$

$i = 1, 2$. In Algorithm 2.2, therefore, $u^n|_{\gamma_1} = u_2^n$ and $u^n|_{\gamma_2} = u_1^n$. Second, setting $u_i^0 = u^0|_{M_i}$, Algorithm 2.2 then splits into two independent iterative processes:

$$u_2^0, u_1^1, u_2^2, u_1^3, u_2^4, u_1^5, \dots$$

and

$$u_1^0, u_2^1, u_1^2, u_2^3, u_1^4, u_2^5, \dots$$

Each process is a Schwarz alternating procedure. Either one of the two is sufficient for the purpose of approximation.

2.3. Convergence Theory. Algorithms 2.1 and 2.2 converge when f, φ, u and u^0 have some mild regularities.

When M is a Euclidean domain and $\varphi = 0$ in (2.2), the H^1 -convergence of Algorithm 2.1 had been proved by P. L. Lions in [47, Theorem. I.2]. The proof was generalized to the case of manifolds in [12, Theorem 2.1].

THEOREM 2.3 ([47, 12] Convergence of Algorithm 2.1). *Under Assumption 2.1, suppose further $f \in H^{-1}(M)$ and $\varphi \in H^{\frac{1}{2}}(\partial M)$ in (2.2). Suppose $u^0 \in H^1(M)$ in Algorithm 2.1. Let u^n be the one in Algorithm 2.1. Then the following holds:*

- (1) For each $n > 0$, $u^n \in H^1(M)$.
- (2) There exist constants $C_0 > 0$ and $C \in [0, 1)$ such that

$$\|u - u^n\|_{H^1(M)} \leq C_0 C^n \|u - u^0\|_{H^1(M)},$$

where C_0 and C are independent of u, u^0 and n .

Proof. This has been proved in [12, Theorem 2.1] when $\varphi = 0$. In general, since $\varphi \in H^{\frac{1}{2}}(\partial M)$, there exists a $v \in H^1(M)$ such that $v|_{\partial M} = \varphi$. Now $u - v$ solves the following problem with homogeneous boundary value

$$(2.4) \quad \begin{cases} L(u - v) = f - Lv, \\ (u - v)|_{\partial M} = 0, \end{cases}$$

where $f - Lv \in H^{-1}(M)$. Applying Algorithm 2.1 to (2.4), we obtain an iterative sequence $\{u^n - v\}_{n=1}^\infty$. By [12], the desired conclusion holds for $\{u^n - v\}_{n=1}^\infty$. Since

$$u^n = (u^n - v) + v \quad \text{and} \quad u - u^n = (u - v) - (u^n - v),$$

the conclusion of this theorem follows. $\square$

The following theorem was proved in [62, Theorem 2.4].

THEOREM 2.4 ([62] Convergence of Algorithm 2.2). *Under Assumptions 2.1 and 2.2, assume further all ρ_i 's are Lipschitz continuous. Suppose $\bigcup_{i \neq j}(\gamma_i \cap \gamma_j)$ has a finite $(d-2)$ -dimensional Hausdorff measure. Suppose $f \in H^{-1}(M)$, $u \in C^0(M) \cap H^1(M)$, and $\varphi \in C^0(\partial M) \cap H^{\frac{1}{2}}(\partial M)$. Suppose $u^0 \in C^0(M) \cap H^1(M)$ in Algorithm 2.2. In addition, u^0 and u are assumed Lipschitz continuous if $\partial M \neq \emptyset$. Let u^n be the one in Algorithm 2.2. Then the following holds:*

- (1) For each $n > 0$ and each i , $u^n \in C^0(M) \cap H^1(M)$ and $u_i^n \in C^0(M_i) \cap H^1(M_i)$.
- (2) $\lim_{n \rightarrow \infty} \|u_i^n - u\|_{C^0(M_i)} = 0$ and $\lim_{n \rightarrow \infty} \|u_i^n - u\|_{H^1(M_i)} = 0$ for each i .
- (3) $\lim_{n \rightarrow \infty} \|u^n - u\|_{C^0(M)} = 0$ and $\lim_{n \rightarrow \infty} \|u^n - u\|_{H^1(M)} = 0$.

REMARK 2.5. In [12] and [62], the problem (2.2) was numerically solved by finite element methods (FEMs). Following the convention of FEMs, the f in (2.2) was assumed to have slightly higher regularity in [12] and [62], more precisely, $f \in L^2(M)$. Typically, FEMs result in a linear system $AX = b$, where A is a stiffness matrix and b comes from f . If $f \in L^2(M)$, then b can be easily obtained via numerical integration. Nevertheless, the proofs of Theorems 2.3 and 2.4 go through even if we merely assume $f \in H^{-1}(M)$.

3. Numerical Scheme. In this section, we propose numerical DDM iterative procedures (Algorithms 3.1 and 3.2 below) to obtain approximations to the solution of (2.2). They are numerical imitations of Algorithms 2.1 and 2.2 based on physics-informed neural networks (PINNs). The idea is as follow. First, M is decomposed into overlapping subdomains M_i ($1 \leq i \leq m$), and each M_i is in a coordinate chart. Theoretically, we can apply Algorithms 2.1 and 2.2 by virtue of this decomposition. Second, since M_i is in a coordinate chart, each subproblem in M_i can be naturally converted to one in a domain D_i in a Euclidean space. In practice, we may use PINNs method to solve problems in D_i approximately. The combination of these two ingredients yields our numerical algorithms.

3.1. Domain Decompositions. Let M be a d -dimensional compact Riemannian manifold with or without boundary. Suppose M is decomposed as Assumption 2.1, i.e. $M = \bigcup_{i=1}^m (M_i \setminus \overline{\gamma_i})$, and each ∂M_i is Lipschitz. We can further require that there is a smooth (i.e. C^∞) diffeomorphism $\phi_i: D_i \rightarrow M_i$ for each i , where D_i is a domain in $\mathbb{R}^d$ with simple shape and Lipschitz boundary.

Theoretically, we can always get such triples (M_i, D_i, ϕ_i) 's. By the definition of a manifold, for each $\zeta \in M$, there is an open chart neighborhood U_ζ of ζ , i.e. there is a smooth diffeomorphism $\phi_\zeta: \Omega_\zeta \rightarrow U_\zeta$, where Ω_ζ is an open subset of $\mathbb{R}_+^d$. Here $\mathbb{R}_+^d$ is a closed half Euclidean space

$$\mathbb{R}_+^d = \{(x_1, \dots, x_d) \in \mathbb{R}^d \mid x_d \geq 0\}.$$

If $\zeta \in M \setminus \partial M$, particularly $\partial M = \emptyset$, we can even require that Ω_ζ is open in $\mathbb{R}^d$. Since Ω_ζ is open in $\mathbb{R}_+^d$ and $\phi_\zeta^{-1}(\zeta) \in \Omega_\zeta$, we can choose a neighborhood D_ζ of $\phi_\zeta^{-1}(\zeta)$ in $\mathbb{R}_+^d$ such that $D_\zeta \subseteq \Omega_\zeta$ and D_ζ is a compact domain with simple shape

and Lipschitz boundary. For instance, D_ζ can be chosen as (d -dimensional) cubes, balls and polydisks, among others. This yields a diffeomorphism $\phi_\zeta: D_\zeta \rightarrow M_\zeta \subset U_\zeta$, where M_ζ is a neighborhood of ζ . The interiors of all such M_ζ 's provide an open cover of M . Since it is compact, M can be covered by the interiors of finitely many such M_ζ 's. These finitely many $(M_\zeta, D_\zeta, \phi_\zeta)$'s yield a desired decomposition of M . In the above, the interior of M_ζ is in the sense of point set topology. More precisely, let $\gamma_\zeta = M_\zeta \cap (M \setminus \partial M)$, then the interior of M_ζ is $M_\zeta \setminus \overline{\gamma_\zeta}$. Each $M_\zeta \setminus \overline{\gamma_\zeta}$ is an open subset of M . If $M_\zeta \cap \partial M = \emptyset$, particularly $\partial M = \emptyset$, then $\overline{\gamma_\zeta} = \partial M_\zeta$.

Suppose we have the desired triples (M_i, D_i, ϕ_i) 's for M . Since an elliptic problem in M_i can be converted to one in D_i via ϕ_i , for the implement of Algorithms 2.1 and 2.2, we merely need to solve problems in these D_i 's. However, to update the boundary values on the ∂D_i 's, transmission of data among different D_i 's is needed.

For $i \neq j$, let $D_{ij} = \phi_i^{-1}(M_i \cap M_j) \subseteq D_i$ and $D_{ji} = \phi_j^{-1}(M_i \cap M_j) \subseteq D_j$. (See Fig. 1 for an illustration, where the whale shaped object is a manifold M , the square and the disk stand for D_i and D_j , respectively, and the shadowed parts of D_i and D_j are D_{ij} and D_{ji} , respectively.) If $M_i \cap M_j \neq \emptyset$, then

$$\phi_j^{-1} \circ \phi_i: D_{ij} \rightarrow D_{ji}$$

is a diffeomorphism which is the transition of coordinates on the overlapping between M_i and M_j . Suppose $\phi_j^{-1} \circ \phi_i$ maps some part of ∂D_i into $D_{ji} \subseteq D_j$, a desired function value on D_j is transported to ∂D_i via this transition of coordinates.

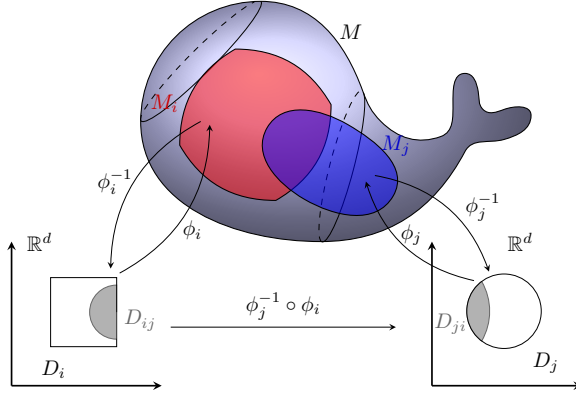


FIG. 1. An illustration of domain decompositions.

It's worth to noting that, as pointed in [12, §4], if manifolds M and M' already have desired decompositions, a good decomposition of the product manifold $M \times M'$ is automatically obtained. More precisely, suppose M (resp. M') has a decomposition $\{(M_i, D_i, \phi_i) \mid 1 \leq i \leq m\}$ (resp. $\{(M'_{i'}, D'_{i'}, \phi'_{i'}) \mid 1 \leq i' \leq m'\}$), then $M \times M'$ has the decomposition

$$\{(M_i \times M'_{i'}, D_i \times D'_{i'}, \phi_i \times \phi'_{i'}) \mid 1 \leq i \leq m, 1 \leq i' \leq m'\}.$$

The transition of coordinates between $D_i \times D'_{i'}$ and $D_j \times D'_{j'}$, is

$$(\phi_j \times \phi'_{j'})^{-1} \circ (\phi_i \times \phi'_{i'}) = (\phi_j^{-1} \circ \phi_i) \times (\phi'_{j'}^{-1} \circ \phi'_{i'}).$$

This canonical technique of decomposition for product manifolds will be employed in our numerical test (see §4.3).

3.2. Numerical Algorithms. Now we propose our numerical DDMs to solve (2.2). The following assumption is needed.

ASSUMPTION 3.1. *Suppose $M = \bigcup_{i=1}^m (M_i \setminus \overline{\gamma_i})$. There is a C^∞ diffeomorphism $\phi_i: D_i \rightarrow M_i$ for each i , where D_i is a compact domain in $\mathbb{R}^d$ with Lipschitz boundary.*

By (2.3), the restriction of (2.2) to M_i is transformed via ϕ_i to the following equation in D_i , where $\hat{u}_i = u \circ \phi_i$.

$$(3.1) \quad \hat{L}\hat{u}_i := - \sum_{\alpha, \beta=1}^d g^{\alpha\beta} \frac{\partial^2 \hat{u}_i}{\partial x_\alpha \partial x_\beta} - \frac{1}{\sqrt{G}} \sum_{\beta=1}^d \sum_{\alpha=1}^d \frac{\partial}{\partial x_\alpha} \left(g^{\alpha\beta} \sqrt{G} \right) \frac{\partial \hat{u}_i}{\partial x_\beta} + b\hat{u}_i = f \circ \phi_i.$$

We shall solve (3.1) with suitable boundary values in D_i by physics-informed neural networks (PINNs) method. The implementation of the PINNs method requires taking high-order derivatives of the neural networks functions. The activation functions, therefore, should possess C^k regularity for sufficiently large k . Fixing a C^k activation function and a neural network in D_i , let V_i be the function space associated with this neural network. Then $V_i \subset C^k(D_i)$. Note that V_i is not a linear subspace of $C^k(D_i)$ since it is not closed under addition. Nevertheless, all elements of V_i are represented by a fixed finite number, say N , of parameters. Therefore, V_i is the image of a continuous map from $\mathbb{R}^N$ to $C^k(D_i)$.

Define a product space

$$V := \prod_{i=1}^m V_i \subset \prod_{i=1}^m C^k(D_i).$$

We propose the following Algorithm 3.1 which is a numerical imitation of Algorithm 2.1.

Algorithm 3.1 Numerical sequential DDM with PINNs to solve (2.2), under Assumption 3.1.

- 1: Choose an initial guess $\hat{u}^0 = (\hat{u}_1^0, \dots, \hat{u}_m^0) \in V$.
- 2: For each $n > 0$,
 for $1 \leq i \leq m$,
 (assuming $\hat{u}^{n-1}$ and $\hat{u}_j^n$ have been obtained for all $j < i$.) define
 a function $\hat{\varphi}_i^n$ on ∂D_i as follows. For $\xi \in \partial D_i$, if $\phi_i(\xi) \in \partial M$,
 let $\hat{\varphi}_i^n(\xi) = \varphi(\phi_i(\xi))$. Otherwise, if $\{j \mid j < i, \phi_i(\xi) \in M_j\} \neq \emptyset$,
 then let j_0 be the maximum of this set and define

$$\hat{\varphi}_i^n(\xi) = \hat{u}_{j_0}^n(\phi_{j_0}^{-1} \circ \phi_i(\xi)).$$

Otherwise, let $j_0 = \max\{j \mid j \neq i, \phi_i(\xi) \in M_j\}$ and define

$$\hat{\varphi}_i^n(\xi) = \hat{u}_{j_0}^{n-1}(\phi_{j_0}^{-1} \circ \phi_i(\xi)).$$

Use PINNs method to find a function $\hat{u}_i^n \in V_i$ which approximates the solution to (3.1) with boundary value $\hat{\varphi}_i^n$.

Define $\hat{u}^n = (\hat{u}_1^n, \dots, \hat{u}_m^n) \in V$.

By Assumption 3.1, we have $\phi_i(\partial D_i) = \partial M_i \subset \partial M \cup \bigcup_{j \neq i} M_j$. If $\xi \in \partial D_i$ and $\phi_i(\xi) \notin \partial M$, then

$$\{j \mid j \neq i, \phi_i(\xi) \in M_j\} \neq \emptyset.$$

So the $\hat{\varphi}_i^n$ in Algorithm 3.1 is well-defined.

Obviously, the $\hat{u}_i^n$ in Algorithm 3.1 is a numerical imitation of the u_i^n in Algorithm 2.1. Besides its numerical nature, Algorithm 3.1 is remarkably different from Algorithm 2.1 in two aspects. First, the u_i^n in Algorithm 2.1 is globally defined on M , but the $\hat{u}_i^n$ in Algorithm 3.1 is only locally defined on D_i . Second, the u^n in Algorithm 2.1 is also globally defined on M , however, $\hat{u}^n$ is a group of local functions. In fact, coordinates are indispensable for numerical computations. Since there are no global coordinates on M , it's impossible to express functions concretely over the entire M in practice. To solve (2.2) numerically, a feasible strategy is to approximate the exact solution u piecewisely on a family of charts $\{\phi_i(D_i) \mid 1 \leq i \leq m\}$ which cover M . The approximation of $\hat{u}_i^n$ to $u \circ \phi_i$ is equivalent to the approximation of $\hat{u}_i^n \circ \phi_i^{-1}$ to $u|_{\phi_i(D_i)}$. If $\hat{u}_i^n$ approximates $u \circ \phi_i$ well for each i , the goal of numerical computation is achieved. This strategy had also been used in [63, 12, 62] where finite element methods rather than PINNs were applied. In this paper, our numerical experiments in §4 show that this approach also works well for PINNs.

Now we formulate our parallel numerical DDM which needs a partition of unity.

ASSUMPTION 3.2. *The partition of unity $\{\rho_i \mid 1 \leq i \leq m\}$ is subordinate to the decomposition in Assumption 3.1, i.e. $\text{supp}\rho_i \subset M_i \setminus \overline{\gamma_i}$, where $\text{supp}\rho_i$ is the support of ρ_i , and $M_i = \phi_i(D_i)$. Furthermore, each ρ_i is continuous.*

In practice, the ρ_i 's can be constructed as follows. Choose nonnegative continuous functions μ_i 's on M such that $\text{supp}\mu_i \subset \phi_i(D_i) \setminus \overline{\gamma_i}$ and $\sum_{i=1}^m \mu_i > 0$ on M . A desired ρ_i can be defined as

$$(3.2) \quad \rho_i = \frac{\mu_i}{\sum_{j=1}^m \mu_j}.$$

To obtain μ_i , it suffices to define $\mu_i \circ \phi_i$ which is a function on D_i and hence can be defined simply and quite arbitrarily in terms of elementary functions.

The following Algorithm 3.2 is a numerical imitation of Algorithm 2.2.

Algorithm 3.2 Numerical parallel DDM with PINNs to solve (2.2), under Assumptions 3.1 and 3.2.

1: Choose an initial guess $\hat{u}^0 = (\hat{u}_1^0, \dots, \hat{u}_m^0) \in V$.

2: For each $n > 0$,

for $1 \leq i \leq m$,

define a function $\hat{\varphi}_i^n$ on ∂D_i as follows. For $\xi \in \partial D_i$, if $\phi_i(\xi) \in \partial M$, let $\hat{\varphi}_i^n(\xi) = \varphi(\phi_i(\xi))$. Otherwise, define

$$(3.3) \quad \hat{\varphi}_i^n(\xi) = \sum_{j=1}^m \rho_j(\phi_i(\xi)) \cdot \hat{u}_j^{n-1}(\phi_j^{-1} \circ \phi_i(\xi)).$$

Use PINNs method to find a function $\hat{u}_i^n \in V_i$ which approximates the solution to (3.1) with boundary value $\hat{\varphi}_i^n$.

Define $\hat{u}^n = (\hat{u}_1^n, \dots, \hat{u}_m^n) \in V$.

The sum in (3.3) needs more explanations. If $\phi_i(\xi) \notin \phi_j(D_j)$, then $\phi_j^{-1} \circ \phi_i(\xi)$ is undefined. However, in this situation, $\rho_j(\phi_i(\xi)) = 0$ and hence the summand $\rho_j(\phi_i(\xi)) \cdot \hat{u}_j^{n-1}(\phi_j^{-1} \circ \phi_i(\xi))$ is actually redundant. So (3.3) indeed makes sense. Furthermore, if $\phi_i(\xi) \in \phi_j(D_j)$, then

$$\rho_j(\phi_i(\xi)) = (\rho_j \circ \phi_j)(\phi_j^{-1} \circ \phi_i(\xi)).$$

Here $\rho_j \circ \phi_j$ is a function on D_j with an explicit formula. Thus $\rho_j(\phi_i(\xi))$ can be obtained once we get the coordinates of $\phi_j^{-1} \circ \phi_i(\xi)$.

Like those in Algorithm 3.1, each $\hat{u}_i^n$ in Algorithm 3.2 is a numerical imitation of the u_i^n in Algorithm 2.2, and each $\hat{u}_i^n \circ \phi_i^{-1}$ is expected to approximate the exact solution u of (2.2) on $\phi(D_i)$. However, unlike those in Algorithm 3.1, the $\hat{\varphi}_i^n$'s and $\hat{u}_i^n$'s for $1 \leq i \leq m$ in Algorithm 3.2 can be obtained independently and parallelly at each iterative step n .

Similar to Algorithms 2.1 and 2.2, when M is decomposed into two subdomains, Algorithms 3.1 and 3.2 are both reduced to Schwarz Alternating Methods. The reason is the same as the one explained in §2.2. The following Algorithm 3.3 is a numerical Schwarz Alternating Method on manifolds.

Algorithm 3.3 Numerical Schwarz Alternating Method with PINNs to solve (2.2), under Assumption 3.1 with $m = 2$.

- 1: Choose an initial guess functions $\hat{u}_2^0$ on D_2 .
- 2: For each $n > 0$,
 - define a function $\hat{\varphi}_1^n$ on ∂D_1 as follows. For $\xi \in \partial D_1$, if $\phi_1(\xi) \in \partial M$, let $\hat{\varphi}_1^n(\xi) = \varphi(\phi_1(\xi))$. Otherwise, define

$$\hat{\varphi}_1^n(\xi) = \hat{u}_2^{n-1}(\phi_2^{-1} \circ \phi_1(\xi)).$$

Use PINNs method to find a function $\hat{u}_1^n \in V_1$ which approximates the solution to (3.1) with boundary value $\hat{\varphi}_1^n$.

Define a function $\hat{\varphi}_2^n$ on ∂D_2 as follows. For $\xi \in \partial D_2$, if $\phi_2(\xi) \in \partial M$, let $\hat{\varphi}_2^n(\xi) = \varphi(\phi_2(\xi))$. Otherwise, define

$$\hat{\varphi}_2^n(\xi) = \hat{u}_1^n(\phi_1^{-1} \circ \phi_2(\xi)).$$

Use PINNs method to find a function $\hat{u}_2^n \in V_2$ which approximates the solution to (3.1) with boundary value $\hat{\varphi}_2^n$.

Note that PINNs method essentially solves a problem of optimization. Typically, a search for the optimal function of this problem is an iterative process. As a result, the above Algorithms 3.1, 3.2 and 3.3 are nested iterations. The outer iterations are frameworks provided by DDMS. The inner iterations are the implementations of PINNs at each iterative step of the frameworks.

3.3. Physics-Informed Neural Networks. The physics-informed neural networks (PINNs) method popularized by [65] can solve general (linear or nonlinear) differential equations in domains in Euclidean spaces. It has many advantages. A major advantage is its mesh-free property. Another major one is that it has strong ability to mitigate the curse of dimensionality. In this paper, we shall mainly take the second advantage mentioned here. More precisely, we shall solve (3.1) with a certain boundary value $\hat{\varphi}_i^n$ in a domain D_i in $\mathbb{R}^d$, where $d \geq 5$. In the case of such high dimensions, (3.1) can hardly be solved effectively by traditional methods, including finite difference method and finite element method.

There are three main ingredients of PINNs: (1) the architecture of neural networks; (2) a loss function to be optimized; (3) an algorithm of optimization. Let's introduce one by one the ingredients in our case.

First, we shall build two types of deep neural networks on D_i , which will be

described in §4. The activation function is chosen as the hyperbolic tangent function $\tanh$ which is C^∞ and even analytic. As before, let V_i denote the function space associated with a chosen neural network on D_i . Now $V_i \subset C^\infty(D_i)$.

Second, we define the loss function $\mathcal{L}_{PINN}$ for (3.1) with boundary value $\hat{\varphi}_i^n$ as,

$$(3.4) \quad \forall v \in V_i, \quad \mathcal{L}_{PINN}(v) = (1 - \lambda_{BC})\mathcal{L}_{PDE}(v) + \lambda_{BC}\mathcal{L}_{BC}(v),$$

where $\lambda_{BC} \in (0, 1)$ is a positive hyperparameter of penalty. (The abbreviation “BC” stands for “boundary condition”.) In addition,

$$(3.5) \quad \mathcal{L}_{PDE}(v) = \frac{1}{|D_i|} \|\hat{L}v - f \circ \phi_i\|_{L^2(D_i)}^2 = \frac{1}{|D_i|} \int_{D_i} |\hat{L}v(x) - f \circ \phi_i(x)|^2 dx$$

and

$$(3.6) \quad \mathcal{L}_{BC}(v) = \frac{1}{|\partial D_i|} \|v - \hat{\varphi}_i^n\|_{L^2(\partial D_i)}^2 = \frac{1}{|\partial D_i|} \int_{\partial D_i} |v(y) - \hat{\varphi}_i^n(y)|^2 dy,$$

where $|D_i|$ (resp. $|\partial D_i|$) is the d -dimensional (resp. $(d-1)$ -dimensional) volume of D_i (resp. ∂D_i). The above integrals are numerically approximated using Monte Carlo method. We shall solve the following optimization problem

$$(3.7) \quad \hat{u}_i^n := \argmin_{v \in V_i} \mathcal{L}_{PINN}(v).$$

At first glance, (3.7) is a linear least squares problem. But it is actually not because, as mentioned before, V_i is not a linear space. Note that all elements of V_i are represented by a fixed finite number, say N , of parameters. Thus V_i is the image of a continuous map $F: \mathbb{R}^N \rightarrow C^\infty(D_i)$, where F is determined by the architecture of the neural network and is often highly nonlinear. In this context, each $X \in \mathbb{R}^N$ is a collection of parameters with specific values representing a function $F(X) \in V_i$. Now (3.7) is converted to the following down-to-earth optimization problem

$$(3.8) \quad \arg \min_{X \in \mathbb{R}^N} \mathcal{L}_{PINN}(F(X))$$

which is highly nonlinear and nonconvex.

Third, in practice, (3.8) can only be approximately solved. A search of good approximations is usually an iterative process. We shall use Adaptive Moment Estimation (Adam) [42] optimizer to implement this search. Furthermore, to compute (3.5) and (3.6), we shall randomly sample a collection of points $\{x_i\}_{i=1}^{N_p}$ (resp. $\{y_i\}_{i=1}^{N_b}$) in D_i (resp. on ∂D_i). These sampling points are updated at each Adam iterative step in order to avoid overfitting, although the numbers N_p and N_b remain constants.

4. Numerical Experiments. We perform numerical tests of Algorithms 3.1 and 3.2 on manifolds with or without boundary. The dimensions vary from 5 to 10. The manifolds are S^5 , S^{10} , $B^3 \times S^3$ and $\mathbb{CP}^3$. Here S^d is the unit sphere in $\mathbb{R}^{d+1}$, B^d is the unit ball in $\mathbb{R}^d$, and $\mathbb{CP}^3$ is the complex projective space with complex dimension 3 and hence real dimension 6.

4.1. On S^5 . We shall perform numerical tests on 5-dimensional manifold S^5 .

First of all, we introduce a general decomposition on $M = S^d$, where S^d is the unit sphere in $\mathbb{R}^{d+1}$, i.e.

$$M = S^d = \left\{ y = (y_1, \dots, y_{d+1}) \in \mathbb{R}^{d+1} \left| \|y\|^2 = \sum_{i=1}^{d+1} y_i^2 = 1 \right. \right\}.$$

This is a d -dimensional manifold without boundary. We decompose S^d into two subdomains as follows. By stereographic projections from the south pole $(0, \dots, 0, -1)$ and north pole $(0, \dots, 0, 1)$, we obtain two subdomains M_1 and M_2 with coordinates whose interiors form an open cover of S^d . For an illustration please refer to Fig. 2,

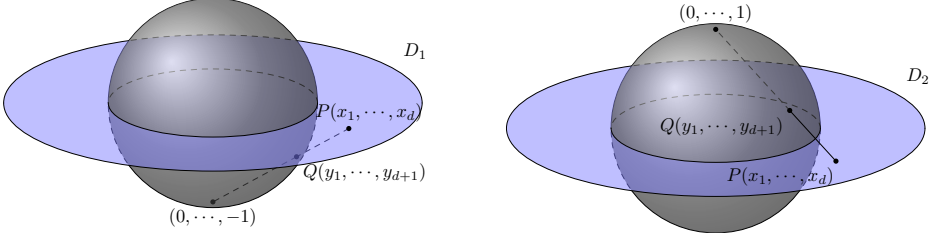


FIG. 2. An illustration of stereographic projections.

where the vertical direction stands for the direction of the $(d+1)$ -th coordinate axis, the disk D_i is a closed ball in

$$\mathbb{R}^d \cong \mathbb{R}^d \times \{0\} \subset \mathbb{R}^{d+1},$$

and the intersection of $\mathbb{R}^d \times \{0\}$ and S^d is the equator of S^d . For each point $P = (x_1, \dots, x_d) \in D_1$, the line segment between P and the south pole $(0, \dots, 0, -1)$ intersects S^d at a single point $Q = (y_1, \dots, y_{d+1})$ other than $(0, \dots, 0, -1)$. The map $P \mapsto Q$ provides an embedding $\phi_1 : D_1 \rightarrow S^d$. We obtain another embedding ϕ_2 if the south pole is replaced with the north pole. More precisely, we have

$$(4.1) \quad D_1 = D_2 = B^d(r) = \{x = (x_1, \dots, x_d) \in \mathbb{R}^d \mid \|x\| \leq r\} \subset \mathbb{R}^d,$$

i.e. D_1 and D_2 are the ball in $\mathbb{R}^d$ with center the origin $\mathbf{0}$ and radius r . The stereographic projections provide diffeomorphisms $\phi_i : D_i \rightarrow M_i$ as

$$(4.2) \quad \begin{aligned} \phi_1 : D_1 &\rightarrow M_1 \subset S^d \subset \mathbb{R}^{d+1} \\ x &\mapsto \left(\frac{2x}{1 + \|x\|^2}, \frac{1 - \|x\|^2}{1 + \|x\|^2} \right) \end{aligned}$$

and

$$(4.3) \quad \begin{aligned} \phi_2 : D_2 &\rightarrow M_2 \subset S^d \subset \mathbb{R}^{d+1} \\ x &\mapsto \left(\frac{2x}{1 + \|x\|^2}, \frac{-1 + \|x\|^2}{1 + \|x\|^2} \right). \end{aligned}$$

To guarantee $S^d = \bigcup_{i=1}^2 (M_i \setminus \overline{\gamma_i}) = \bigcup_{i=1}^2 (M_i \setminus \partial M_i)$, we have to let $r > 1$. The larger r is, the more overlapping there will be. The transitions of coordinates are given by

$$\phi_2^{-1} \circ \phi_1(x) = \phi_1^{-1} \circ \phi_2(x) = \frac{x}{\|x\|^2}.$$

Second, equip S^d with the Riemannian metric g inherited from the standard one on $\mathbb{R}^{d+1}$. On each D_i ,

$$g = 4(1 + \|x\|^2)^{-2} \sum_{\alpha=1}^d dx_\alpha \otimes dx_\alpha,$$

and

$$\Delta v = 4^{-1}(1 + \|x\|^2)^2 \sum_{\alpha=1}^d \frac{\partial^2 v}{\partial x_\alpha^2} - 2^{-1}(d-2)(1 + \|x\|^2) \sum_{\alpha=1}^d x_\alpha \frac{\partial v}{\partial x_\alpha}.$$

Third, consider the model problem (2.2) on S^d with $b > 0$. Choose the exact solution to (2.2) as

$$u = y_{d+1},$$

where y_{d+1} is the $(d+1)$ -th coordinate of $\mathbb{R}^{d+1}$. Then

$$f = (d+b)u$$

in (2.2). On D_i , the exact solution u has the expression

$$\hat{u}_1(x) := u \circ \phi_1(x) = \frac{1 - \|x\|^2}{1 + \|x\|^2} \quad \text{and} \quad \hat{u}_2(x) := u \circ \phi_2(x) = \frac{-1 + \|x\|^2}{1 + \|x\|^2}.$$

In the following, we choose $b = 1$. Take $r = 1.2$ in (4.1). Numerical experiments are performed on S^d with $d = 5$ and 10. Since S^d is decomposed into two subdomains, Algorithms 3.1 and 3.2 are both reduced to Algorithm 3.3. We only need to test Algorithm 3.3 in the current situation.

Our numerical experiments begin with the case S^5 . We employ a fully connected neural network with 4 hidden layers. The width of each hidden layer is 500. The activation function is the hyperbolic tangent function $\tanh$. The λ_{BC} in (3.4) is set to 0.8. For each inner iteration, we sample a total of 2000 random points, with 1200 in the interior and 800 on the boundary. For each outer iterative step, the inner iteration runs 5000 steps using the Adam optimizer.

Define the relative L^2 -error of $\hat{u}^n = (\hat{u}_1^n, \hat{u}_2^n)$ with respect to the exact solution $\hat{u} = (\hat{u}_1, \hat{u}_2)$ as

$$(4.4) \quad \max_{1 \leq i \leq 2} \left\{ \frac{\|\hat{u}_i^n - \hat{u}_i\|_{L^2(D_i)}}{\|\hat{u}_i\|_{L^2(D_i)}} \right\},$$

where $\hat{u}^n$ is the n -th iterative approximation of the outer iteration.

We perform five numerical tests independently. The mean value and standard deviation of the relative L^2 -errors (4.4) are reported in Table 1, where the relative L^2 -errors are calculated by Monte Carlo method.

n	0	5	10	15	20	100
Mean	1.0559	0.2067	0.0290	$4.18e-3$	$1.62e-3$	$3.47e-4$
Std Dev	0.0452	0.0316	$5.53e-3$	$1.34e-3$	$6.59e-4$	$2.77e-5$

TABLE 1
Relative L^2 -errors of $\hat{u}^n$ with respect to $\hat{u}$ on S^5 .

We also plot the convergence logarithm curve in Fig. 3, where the horizontal coordinate is n , the vertical coordinate is the mean of the relative L^2 -errors (4.4).

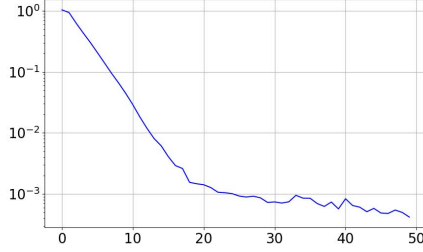
As shown in Table 1 and Fig. 3, the $\hat{u}^n$ sequence approximates the exact solution $\hat{u}$ well, and the outer iteration stabilizes after 60 steps.

We set $\hat{u}^\infty := \hat{u}^{100}$ and define the relative L^2 -error of $\hat{u}^n$ with respect to $\hat{u}^\infty$ as

$$(4.5) \quad \max_{1 \leq i \leq 2} \left\{ \frac{\|\hat{u}_i^n - \hat{u}_i^\infty\|_{L^2(D_i)}}{\|\hat{u}_i^\infty\|_{L^2(D_i)}} \right\}.$$

FIG. 3. *Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}$ on S^5 .*

The convergence logarithm curve of $\hat{u}^n$ with respect to $\hat{u}^\infty$ is in Fig. 4, where the horizontal coordinate is n , the vertical coordinate is the mean of the relative L^2 -errors (4.5).

FIG. 4. *Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}^\infty$ on S^5 .*

4.2. On S^{10} . Now we perform numerical experiments on S^{10} . The network is a fully connected one with 6 hidden layers, the width of each hidden layer is 500. The activation function is tanh. The λ_{BC} in (3.4) is set to 0.9. For each inner iteration, we sample a total of 5000 random points, with 4500 in the interior and 500 on the boundary. For each outer iterative step, the inner iteration runs 5000 steps using the Adam optimizer.

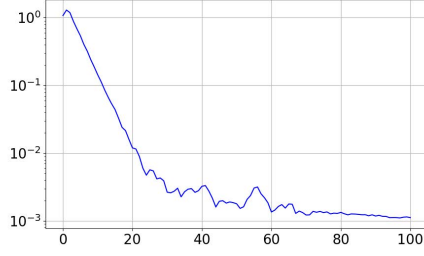
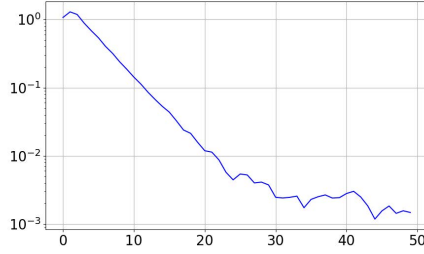
We test five times independently. The numerical results are presented in Table 2 and Figs. 5 and 6. They also show that the $\hat{u}^n$ sequence approximates the exact solution $\hat{u}$ well.

n	0	5	10	15	20	100
Mean	1.1359	0.5412	0.1435	0.0443	0.0131	$1.13e-3$
Std Dev	0.1594	0.2394	0.0600	0.0307	$9.01e-3$	$1.09e-4$

TABLE 2
Relative L^2 -errors of $\hat{u}^n$ with respect to $\hat{u}$ on S^{10} .

4.3. On $B^3 \times S^3$. We shall perform numerical tests on 6-dimensional manifold $B^3 \times S^3$ which has boundary $S^2 \times S^3$.

First of all, we introduce a general decomposition on $M = B^p \times S^q$. Here B^p is

FIG. 5. *Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}$ on S^{10} .*FIG. 6. *Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}^\infty$ on S^{10} .*

the unit ball in $\mathbb{R}^p$, i.e.

$$B^p = \{y = (y_1, \dots, y_p) \in \mathbb{R}^p \mid \|y\| \leq 1\},$$

and S^q is the unit sphere in $\mathbb{R}^{q+1}$. This M is a manifold with dimension $p + q$ and with boundary $\partial M = (\partial B^p) \times S^q = S^{p-1} \times S^q$.

As mentioned at the end of §3.1, to decompose the product manifold $B^p \times S^q$, it suffices to decompose its factors B^p and S^q . Since B^p is already a simple domain in $\mathbb{R}^p$, it does not need any further decomposition. We decompose S^q by stereographic projections again. Two diffeomorphisms $B^q(r) \rightarrow M'_i \subset S^q$ ($i = 1, 2$) are obtained, where M'_i is a domain in S^q , and $B^q(r)$ is the following ball in $\mathbb{R}^q$,

$$B^q(r) = \{x = (x_1, \dots, x_q) \in \mathbb{R}^q \mid \|x\| \leq r\}.$$

In addition, the formulas of these two diffeomorphisms are given by (4.2) and (4.3).

Overall, we can define

$$(4.6) \quad D_1 = D_2 = B^p \times B^q(r) \subset \mathbb{R}^{p+q}.$$

The maps of decomposition of $B^p \times S^q$ are two diffeomorphisms

$$\begin{aligned} \phi_1: D_1 = B^p \times B^q(r) &\rightarrow M_1 \subset M \subset B^p \times \mathbb{R}^{q+1} \\ (y, x) &\mapsto \left(y, \frac{2x}{1 + \|x\|^2}, \frac{1 - \|x\|^2}{1 + \|x\|^2} \right) \end{aligned}$$

and

$$\begin{aligned} \phi_2: D_2 = B^p \times B^q(r) &\rightarrow M_1 \subset M \subset B^p \times \mathbb{R}^{q+1} \\ (y, x) &\mapsto \left(y, \frac{2x}{1 + \|x\|^2}, \frac{-1 + \|x\|^2}{1 + \|x\|^2} \right), \end{aligned}$$

where $y \in B^p$ and $x \in B^q(r)$. Note that, for $i = 1$ and 2 ,

$$\partial D_i = \partial[B^p \times B^q(r)] = [(\partial B^p) \times B^q(r)] \cup [B^p \times \partial B^q(r)].$$

We also see that

$$\gamma_i = \phi_i[(B^p \setminus \partial B^p) \times \partial B^q(r)], \quad \overline{\gamma_i} = \phi_i[B^p \times \partial B^q(r)],$$

and

$$M_i \cap \partial M = \phi_i[(\partial B^p) \times B^q(r)] = \phi_i[S^{p-1} \times B^q(r)].$$

To guarantee $B^p \times S^q = \bigcup_{i=1}^2 (M_i \setminus \overline{\gamma_i})$, we have to let $r > 1$. The larger r is, the more overlapping there will be. The transitions of coordinates are given by

$$\phi_2^{-1} \circ \phi_1(y, x) = \phi_1^{-1} \circ \phi_2(y, x) = \left(y, \frac{x}{\|x\|^2} \right).$$

Second, equip $B^p \times S^q$ with the Riemannian metric g inherited from the standard one on $\mathbb{R}^{p+q+1}$. On each D_i ,

$$g = \sum_{\alpha=1}^p dy_\alpha \otimes dy_\alpha + 4(1 + \|x\|^2)^{-2} \sum_{\alpha=1}^q dx_\alpha \otimes dx_\alpha,$$

and

$$\Delta v = \sum_{\alpha=1}^p \frac{\partial^2 v}{\partial y_\alpha^2} + 4^{-1}(1 + \|x\|^2)^2 \sum_{\alpha=1}^q \frac{\partial^2 v}{\partial x_\alpha^2} - 2^{-1}(q-2)(1 + \|x\|^2) \sum_{\alpha=1}^q x_\alpha \frac{\partial v}{\partial x_\alpha}.$$

Third, consider the model problem (2.2) on $B^p \times S^q$ with $b \geq 0$. Choose the exact solution to (2.2) as

$$u(y, y') = \sin(\pi y_p) + y'_{q+1},$$

where $y = (y_1, \dots, y_p) \in B^p$ and $y' = (y'_1, \dots, y'_{q+1}) \in S^q \subset \mathbb{R}^{q+1}$. Then

$$f = (b + \pi^2) \sin(\pi y_p) + (b + q) y'_{q+1}$$

in (2.2). On D_i , u and f has the expression

$$\hat{u}_1(y, x) := u \circ \phi_1(y, x) = \sin(\pi y_p) + \frac{1 - \|x\|^2}{1 + \|x\|^2},$$

$$\hat{u}_2(y, x) := u \circ \phi_2(y, x) = \sin(\pi y_p) + \frac{-1 + \|x\|^2}{1 + \|x\|^2},$$

$$f \circ \phi_1(y, x) = (b + \pi^2) \sin(\pi y_p) + (b + q) \frac{1 - \|x\|^2}{1 + \|x\|^2},$$

and

$$f \circ \phi_2(y, x) = (b + \pi^2) \sin(\pi y_p) + (b + q) \frac{-1 + \|x\|^2}{1 + \|x\|^2}.$$

Since $B^p \times S^q$ is decomposed into two subdomains, we also only need to test Algorithm 3.3. Now we choose $p = q = 3$ and $b = 0$. Take $r = 1.2$ in (4.6).

We build a new type of network with 4 hidden layers, each of width 500. This network is a variant of ResNet, but it differs considerably from the standard ResNet

and closely resembles the one described in [74, §4]. The hidden part of our network consists of two successive blocks, each of which has two layers but includes only one shortcut connection. More precisely, a function associated with our network takes the form

$$F(X) = W_3 \cdot R_2 \circ R_1(X) + b_3$$

where W_3 is a weight vector and b_3 is a bias scalar. For $i = 1, 2$, the map $R_i: \mathbb{R}^n \rightarrow \mathbb{R}^m$ is defined by

$$R_i(X) = \sigma(W_{i,2}\sigma(W_{i,1}X + b_{i,1}) + b_{i,2} + W_{i,0}X),$$

where σ is an activation function, $W_{i,j}$'s are weight matrices, and $b_{i,j}$'s are bias vectors. Each $R_i(X)$ corresponds to the map represented by one block of the network. The addition of $W_{i,0}X$ is a shortcut connection. We set $W_{i,0} = I$ when $n = m$; in that case, the shortcut reduces to the identity shortcut used in the standard ResNet.

We take our activation function as $\tanh$. The λ_{BC} in (3.4) is set to 0.995. For each inner iteration, we sample a total of 2000 random points, with 200 in the interior and 1800 on the boundary. For each outer iterative step, the inner iteration runs 5000 steps using the Adam optimizer. We perform five tests independently. The numerical results are presented in Table 3 and Figs. 7 and 8.

n	0	5	10	15	20	100
Mean	1.0265	$2.25e-3$	$1.62e-3$	$1.04e-3$	$7.90e-4$	$1.69e-4$
Std Dev	0.0204	$2.91e-4$	$3.95e-4$	$1.03e-4$	$1.02e-4$	$6.66e-6$

TABLE 3
Relative L^2 -errors of $\hat{u}^n$ with respect to $\hat{u}$ on $B^3 \times S^3$.

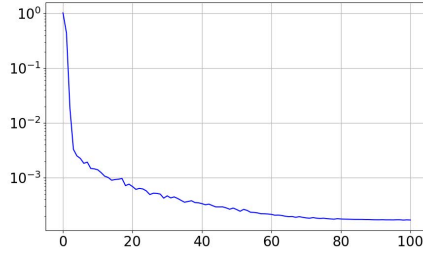


FIG. 7. Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}$ on $B^3 \times S^3$.

4.4. On $\mathbb{CP}^3$: Sequential Domain Decomposition Method. We shall test Algorithm 3.1 on the complex projective space $\mathbb{CP}^3$ which has complex dimension 3 and real dimension 6. Unlike the previous examples S^d and $B^p \times S^q$, the complex projective space $\mathbb{CP}^k$ is not defined as a submanifold of $\mathbb{R}^n$.

First, let's recall the definition of the complex projective space $\mathbb{CP}^k$ and formulate a canonical decomposition on it. The $\mathbb{CP}^k$ can be defined as a quotient space. Let

$$\mathbb{C}^{k+1} \setminus \{\mathbf{0}\} = \{(w_0, w_1, \dots, w_k) \mid \mathbf{0} \neq (w_0, w_1, \dots, w_k) \in \mathbb{C}^{k+1}\}.$$

Here $\mathbf{0} \in \mathbb{C}^{k+1}$ is the origin, each w_j is a complex number for $0 \leq j \leq k$, and, following the convention of algebraic geometry, the index j starts from 0 rather than 1. Define



FIG. 8. Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}^\infty$ on $B^3 \times S^3$.

a relation of equivalence on $\mathbb{C}^{k+1} \setminus \{\mathbf{0}\}$ as

$$(w_0, w_1, \dots, w_k) \sim (w'_0, w'_1, \dots, w'_k)$$

if and only if

$$(w_0, w_1, \dots, w_k) = \lambda(w'_0, w'_1, \dots, w'_k)$$

for some $0 \neq \lambda \in \mathbb{C}$. Define

$$\mathbb{CP}^k = (\mathbb{C}^{k+1} \setminus \{\mathbf{0}\}) / \sim.$$

Thus, every $P \in \mathbb{CP}^k$ can be represented by a vector $(w_0, w_1, \dots, w_k) \in \mathbb{C}^{k+1} \setminus \{\mathbf{0}\}$. Conventionally, we write

$$P = [w_0, w_1, \dots, w_k],$$

where $[w_0, w_1, \dots, w_k]$ are called the *homogeneous coordinates* of P . Note that, for $\lambda \neq 0$,

$$[w_0, w_1, \dots, w_k] = [\lambda w_0, \lambda w_1, \dots, \lambda w_k].$$

The $\mathbb{CP}^k$ is a compact complex manifold without boundary. It has complex dimension k and hence real dimension $2k$. For more details of $\mathbb{CP}^k$, see [30, p. 15].

Now we decompose $\mathbb{CP}^k$ into $k+1$ subdomains M_j for $0 \leq j \leq k$. In the following, $z_j = x_j + \sqrt{-1}y_j \in \mathbb{C}$, $x_j \in \mathbb{R}$, and $y_j \in \mathbb{R}$. We shall identify the complex number z_j with the 2-dimensional real vector (x_j, y_j) . Let

$$B^2(r) = \{z \in \mathbb{C} \mid |z| \leq r\} = \{(x, y) \in \mathbb{R}^2 \mid \|(x, y)\| \leq r\}.$$

This is a disk in $\mathbb{C} = \mathbb{R}^2$ with center $\mathbf{0}$ and radius r . For $0 \leq j \leq k$, define

$$(4.7) \quad D_j = \prod_{h=0, h \neq j}^k B^2(r) = \{(z_0, \dots, \widehat{z}_j, \dots, z_k) \mid \forall h, z_h \in B^2(r)\} \subset \mathbb{C}^k = \mathbb{R}^{2k},$$

where $\widehat{z}_j$ means z_j is missing. Each D_j is a polydisk with real dimension $2k$. We have the following $k+1$ diffeomorphisms

$$\begin{aligned} \phi_j: D_j &\rightarrow M_j \subset \mathbb{CP}^k \\ (z_0, \dots, \widehat{z}_j, \dots, z_k) &\mapsto [z_0, \dots, z_{j-1}, 1, z_{j+1}, \dots, z_k]. \end{aligned}$$

To guarantee $\mathbb{CP}^k = \bigcup_{j=0}^k (M_j \setminus \partial M_j)$, we have to let $r > 1$. The larger r is, the more overlapping there will be.

The transitions of coordinates are as follows. If $j < h$, then

$$\phi_h^{-1} \circ \phi_j(z_0, \dots, \widehat{z}_j, \dots, z_k) = \frac{1}{z_h}(z_0, \dots, z_{j-1}, 1, z_{j+1}, \dots, \widehat{z}_h, \dots, z_k)$$

and

$$\phi_j^{-1} \circ \phi_h(z_0, \dots, \widehat{z}_h, \dots, z_k) = \frac{1}{z_j}(z_0, \dots, \widehat{z}_j, \dots, z_{h-1}, 1, z_{h+1}, \dots, z_k).$$

Second, equipping it with the classical *Fubini-Study metric* (c.f. [30, p. 30]), $\mathbb{CP}^k$ becomes a Kähler manifold with Kähler form

$$\frac{\sqrt{-1}}{2} \partial \bar{\partial} \log \sum_{j=0}^k |w_j|^2,$$

where $[w_0, w_1, \dots, w_k]$ are the homogeneous coordinates of $\mathbb{CP}^k$. The *Fubini-Study metric*, denoted by $\mathcal{H}$, is a Hermitian metric. On each D_j , it is expressed as

$$\mathcal{H} = (1 + \|z\|^2)^{-1} \sum_{\alpha=0, \alpha \neq j}^k dz_\alpha \otimes d\bar{z}_\alpha - (1 + \|z\|^2)^{-2} \sum_{\alpha=0, \alpha \neq j}^k \sum_{\beta=0, \beta \neq j}^k \bar{z}_\alpha z_\beta dz_\alpha \otimes d\bar{z}_\beta,$$

where $z = (z_0, \dots, \widehat{z}_j, \dots, z_k) \in D_j$,

$$\|z\|^2 = \sum_{\alpha=0, \alpha \neq j}^k |z_\alpha|^2 = \sum_{\alpha=0, \alpha \neq j}^k (x_\alpha^2 + y_\alpha^2).$$

We choose the Riemannian metric g on $\mathbb{CP}^k$ as the real part of $\mathcal{H}$. This g provides the underlying Riemannian structure of the above Kähler structure. The Laplacian is expressed as

$$\begin{aligned} \Delta v &= 2\Delta_\partial v = 2\Delta_{\bar{\partial}} v \\ &= 4(1 + \|z\|^2) \left(\sum_{\alpha=0, \alpha \neq j}^k \frac{\partial^2 v}{\partial z_\alpha \partial \bar{z}_\alpha} + \sum_{\alpha=0, \alpha \neq j}^k \sum_{\beta=0, \beta \neq j}^k z_\alpha \bar{z}_\beta \frac{\partial^2 v}{\partial z_\alpha \partial \bar{z}_\beta} \right) \\ &= (1 + \|z\|^2) \sum_{\alpha=0, \alpha \neq j}^k \left(\frac{\partial^2 v}{\partial x_\alpha^2} + \frac{\partial^2 v}{\partial y_\alpha^2} \right) \\ &\quad + (1 + \|z\|^2) \sum_{\alpha=0, \alpha \neq j}^k \sum_{\beta=0, \beta \neq j}^k \left[(x_\alpha x_\beta + y_\alpha y_\beta) \left(\frac{\partial^2 v}{\partial x_\alpha \partial x_\beta} + \frac{\partial^2 v}{\partial y_\alpha \partial y_\beta} \right) \right. \\ &\quad \left. + (x_\alpha y_\beta - y_\alpha x_\beta) \left(\frac{\partial^2 v}{\partial x_\alpha \partial y_\beta} - \frac{\partial^2 v}{\partial y_\alpha \partial x_\beta} \right) \right]. \end{aligned}$$

Third, we consider the model problem (2.2) with $b > 0$. Choose constants $a_j \in \mathbb{R}$, $0 \leq j \leq k$. Choose the exact solution to (2.2) as

$$(4.8) \quad u([w_0, w_1, \dots, w_k]) = \sum_{j=0}^k a_j |w_j|^2,$$

where $[w_0, \dots, w_k]$ are homogeneous coordinates with normalization $\sum_{j=0}^k |w_j|^2 = 1$. It is easy to see that u is well-defined. The f in (2.2) is then

$$f = (4k + 4 + b)u - 4 \sum_{j=0}^k a_j.$$

On D_j , the exact solution u has the expression

$$\hat{u}_j := u \circ \phi_j(z_0, \dots, \hat{z}_j, \dots, z_k) = \frac{a_j + \sum_{\beta=0, \beta \neq j}^k a_\beta |z_\beta|^2}{1 + \sum_{\beta=0, \beta \neq j}^k |z_\beta|^2}.$$

We perform numerical test on $\mathbb{CP}^3$. Choose b in (2.2) as 4, choose in (4.8)

$$(a_0, a_1, a_2, a_3) = (1, 2, -1, -2).$$

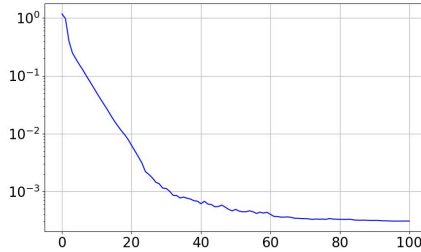
Take $r = 1.2$ in (4.7). Since $\mathbb{CP}^3$ is decomposed into 4 subdomains, Algorithms 3.1 and 3.2 are essentially different in this case. Neither of them is reduced to Algorithm 3.3. We apply Algorithm 3.1 first.

The network architecture is identical to that in §4.3. The λ_{BC} in (3.4) is set to 0.997. For each inner iteration, we sample a total of 2000 random points, with 1000 in the interior and 1000 on the boundary. For each outer iterative step, the inner iteration runs 5000 steps using the Adam optimizer. We perform five tests independently. The numerical results are presented in Table 4 and Figs. 9 and 10. They show the $\hat{u}^n$ sequence approximates the exact solution well.

n	0	5	10	15	20	100
Mean	1.2793	0.1563	0.0508	0.0170	$6.32e-3$	$3.19e-4$
Std Dev	0.1111	0.0565	0.0181	$6.34e-3$	$1.97e-3$	$2.69e-5$

TABLE 4

Relative L^2 -errors of $\hat{u}^n$ with respect to $\hat{u}$ on $\mathbb{CP}^3$ (Sequential DDM).

FIG. 9. Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}$ on $\mathbb{CP}^3$ (Sequential DDM).

4.5. On $\mathbb{CP}^3$: Parallel Domain Decomposition Method. Finally, we apply Algorithm 3.2 to the above specific problem on $\mathbb{CP}^3$. Unlike Algorithm 3.1, we now need a partition of unity $\{\rho_j \mid 0 \leq j \leq 3\}$ satisfying Assumption 3.2.

We first construct a general $\{\rho_j \mid 0 \leq j \leq k\}$ satisfying Assumption 3.2 on $\mathbb{CP}^k$. As mentioned before, it suffices to define the μ_j in (3.2), which is in turn reduced to

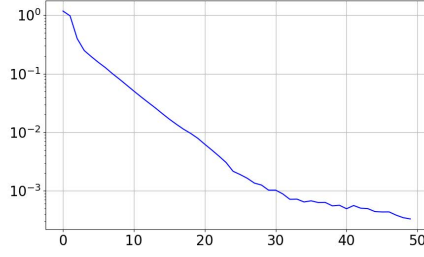


FIG. 10. *Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}^\infty$ on $\mathbb{CP}^3$ (Sequential DDM).*

the definition of $\mu_j \circ \phi_j$ on D_j . Choose $r' \in (1, r)$. Define

$$\mu_j \circ \phi_j(z_0, \dots, \hat{z}_j, \dots, z_k) = \begin{cases} 0, & \max\{|z_h| \mid 0 \leq h \leq k\} > r'; \\ \prod_{h=0, h \neq j}^k (1 - (\frac{|z_h|}{r'})^2)^3, & \text{otherwise.} \end{cases}$$

We see that $\text{supp} \mu_j \subset M_j \setminus \partial M_j$, μ_j is C^2 , and $\sum_{j=0}^k \mu_j > 0$. Hence the resulted family of ρ_j 's are C^2 functions satisfying Assumption 3.2.

Let $r' = 0.1r + 0.9$ and $k = 3$. This yields a desired partition of unity. We keep the above settings for the network architecture, hyperparameters, sampling points, inner iterations, and so forth. The experiment is again repeated independently five times. The numerical results are presented in Table 5 and Figs. 11 and 12.

n	0	5	10	15	20	100
Mean	1.2613	0.1367	0.0439	0.0236	0.0138	$3.21e-4$
Std Dev	0.0724	0.0274	0.0236	0.0142	$8.53e-3$	$1.53e-5$

TABLE 5

Relative L^2 -errors of $\hat{u}^n$ with respect to $\hat{u}$ on $\mathbb{CP}^3$ (Parallel DDM).

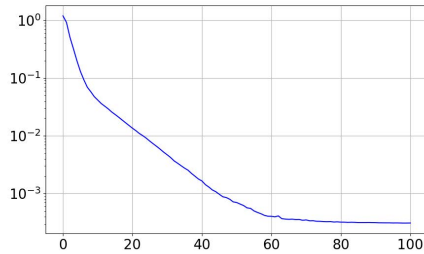


FIG. 11. *Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}$ on $\mathbb{CP}^3$ (Parallel DDM).*

In comparison, the parallel DDM solves the equation on $\mathbb{CP}^3$ with the same accuracy as the sequential DDM; however, the outer iteration of the sequential DDM converges faster than that of the parallel DDM.

Acknowledgements. We thank Jingfei Chen, Jingrun Chen and Tao Zhou for various discussions. The second author was partially supported by NSFC 11871272. The third author was partially supported by NSFC 12371369.

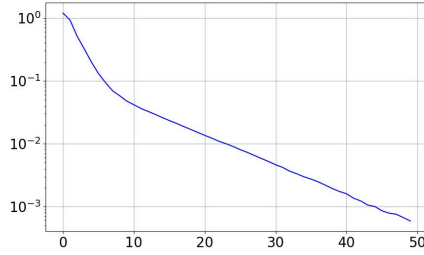


FIG. 12. Relative L^2 -convergence of $\hat{u}^n$ with respect to $\hat{u}^\infty$ on $\mathbb{CP}^3$ (Parallel DDM).

REFERENCES

- [1] H. AI DAAS, L. GRIGORI, P. JOLIVET, AND P. TOURNIER, *A multilevel Schwarz preconditioner based on a hierarchy of robust coarse spaces*, SIAM J. Sci. Comput. 43 (2021), no. 3, A1907–A1928. [2](#)
- [2] M. AINSWORTH AND J. DONG, *Galerkin neural networks: a framework for approximating variational equations with error control*, SIAM J. Sci. Comput. 43 (2021), no. 4, A2474–A2501. [3](#)
- [3] T. AUBIN, *Nonlinear Analysis on Manifolds, Monge-Ampère Equations*, Grundlehren Math. Wiss. 252, Springer-Verlag, New York, 1982. [1](#), [4](#)
- [4] E. BACHINI, M. FARTHING, AND M. PUTTI, *Intrinsic finite element method for advection-diffusion-reaction equations on surfaces*, Journal of Computational Physics, 424 (2021), p. 109827. [1](#)
- [5] J. BARRERA, T. KOLEV, K. MITTAL AND V. TOMOV, *High-order mesh morphing for boundary and interface fitting to implicit geometries*, Comput.-Aided Des. 158 (2023), Paper No. 103499, 11 pp. [1](#)
- [6] A. BARRON, *Universal approximation bounds for superpositions of a sigmoidal function*, IEEE Trans. Inform. Theory 39 (1993), no. 3, 930–945. [3](#)
- [7] J. BAUMGARDNER AND P. FREDERICKSON, *Icosahedral discretization of the two-sphere*, SIAM Journal on Numerical Analysis, 22 (1985), pp. 1107–1115. [1](#)
- [8] A. BONITO, A. DEMLOW, AND M. LICHT, *A divergence-conforming finite element method for the surface Stokes equation*, SIAM Journal on Numerical Analysis, 58 (2020), pp. 2764–2798. [1](#)
- [9] A. BONITO, A. DEMLOW, AND R. NOCHETTO, *Finite element methods for the Laplace–Beltrami operator*, in Handbook of Numerical Analysis, vol. 21, Elsevier, 2020, pp. 1–103. [1](#)
- [10] J. BRAMBLE, J. PASCIAK, J. WANG, AND J. XU, *Convergence estimates for product iterative methods with applications to domain decomposition*, Mathematics of Computation, 57 (1991), pp. 1–21. [2](#)
- [11] X.-C. CAI AND Y. SAAD, *Overlapping domain decomposition algorithms for general sparse matrices*, Numerical Linear Algebra with Applications, 3 (1996), pp. 221–237. [2](#)
- [12] S. CAO AND L. QIN, *A numerical domain decomposition method for solving elliptic equations on manifolds*, SIAM J. Sci. Comput. 46 (2024), no. 1, A376–A398, arXiv:2212.04079. [1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [10](#)
- [13] P. G. CIARLET, *The Finite Element Method for Elliptic Problems*, SIAM, 2002. [2](#)
- [14] G. CYBENKO, *Approximation by superpositions of a sigmoidal function*, Math. Control Signals Systems 2 (1989), no. 4, 303–314. [3](#)
- [15] T. DE RYCK AND S. MISHRA, *Numerical analysis of physics-informed neural networks and related models in physics-informed machine learning*, Acta Numer. 33 (2024), 633–713. [3](#)
- [16] M. DISSANAYAKE AND N. PHAN-THIEN, *Neural-network-based approximations for solving partial differential equations*, Communications in Numerical Methods in Engineering 10 (1994), 195–201. [3](#)
- [17] K. DECKELNICK, G. DZIUK, AND C. ELLIOTT, *Computation of geometric partial differential equations and mean curvature flow*, Acta Numerica, 14 (2005), pp. 139–232. [1](#)
- [18] V. DOBREV, J.-L. GUERMOND, AND B. POPOV, *Surface reconstruction and image enhancement via L^1 -minimization*, SIAM Journal on Scientific Computing, 32 (2010), pp. 1591–1616. [1](#)
- [19] V. DOLEAN, A. HEINLEIN, S. MISHRA AND B. MOSELEY, *Multilevel domain decomposition-based*

- architectures for physics-informed neural networks*, Comput. Methods Appl. Mech. Engrg. 429 (2024), Paper No. 117116. [3](#)
- [20] V. DOLEAN, P. JOLIVET, AND N. NATAF, *An Introduction to Domain Decomposition Methods, Algorithms, Theory, and Parallel Implementation*, Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2015. [2](#)
- [21] M. DRYJA, J. GALVIS, AND M. SARKIS, *A FETI-DP preconditioner for a composite finite element and discontinuous Galerkin method*, SIAM J. Numer. Anal. 51 (2013), no.1, 400–422. [2](#)
- [22] G. DZIUK, *Finite elements for the Beltrami operator on arbitrary surfaces*, Partial Differential Equations and Calculus of Variations, (1988), pp. 142–155. [1](#)
- [23] G. DZIUK, *An algorithm for evolutionary surfaces*, Numerische Mathematik, 58 (1990), pp. 603–611. [1](#)
- [24] G. DZIUK AND C. ELLIOTT, *Finite element methods for surface PDEs*, Acta Numerica, 22 (2013), pp. 289–396. [1](#)
- [25] W. E, *A proposal on machine learning via dynamical systems*, Commun. Math. Stat. 5 (2017), no. 1, 1–11. [3](#)
- [26] W. E, J. HAN AND A. JENTZEN, *Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations*, Commun. Math. Stat. 5 (2017), no. 4, 349–380. [3](#)
- [27] W. E AND B. YU, *The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems*, Commun. Math. Stat. 6 (2018), no. 1, 1–12. [3](#)
- [28] Z. GAO, L. YAN AND T. ZHOU, *Failure-informed adaptive sampling for PINNs*, SIAM J. Sci. Comput. 45 (2023), no. 4, A1971–A1994. [3](#)
- [29] D. GILBARG AND N. TRUDINGER, *Elliptic Partial Differential Equations of Second Order*, Springer-Verlag, Berlin, 2001. [4](#)
- [30] P. GRIFFITHS AND J. HARRIS, *Principles of Algebraic Geometry*, John Wiley & Sons, 2014. [19](#), [20](#)
- [31] L. GUO, H. WU, X. YU AND T. ZHOU, *Monte Carlo fPINNs: deep learning method for forward and inverse problems involving high dimensional fractional partial differential equations*, Comput. Methods Appl. Mech. Engrg. 400 (2022), Paper No. 115523. [3](#)
- [32] J. HAN, A. JENTZEN AND W. E, *Solving high-dimensional partial differential equations using deep learning*, Proc. Natl. Acad. Sci. USA 115 (2018), no. 34, 8505–8510. [3](#)
- [33] J. HE, L. LI, J. XU AND C. ZHENG, *ReLU deep neural networks and linear finite elements*, J. Comput. Math. 38 (2020), no. 3, 502–527. [3](#)
- [34] A. HEINLEIN, A. KLAWONN, M. LANSER AND J. WEBER, *Combining machine learning and domain decomposition methods for the solution of partial differential equations—a review*, GAMM-Mitt. 44 (2021), no. 1, Paper No. e202100001. [3](#)
- [35] M. HOLST, C. MEIER AND G. TSOGTGEREL, *Non-CMC solutions of the Einstein constraint equations on compact manifolds with apparent horizon boundaries*, Comm. Math. Phys. 357 (2018), no. 2, 467–517. [1](#)
- [36] M. HOLST, O. SARBACH, M. TIGLIO AND M. VALLISNERI, *The emergence of gravitational wave science: 100 years of development of mathematical theory, detectors, numerical algorithms, and data analysis tools*, Bull. Amer. Math. Soc. (N.S.) 53 (2016), no.4, 513–554. [1](#)
- [37] M. HOLST AND A. STERN, *Geometric variational crimes: Hilbert complexes, finite element exterior calculus, and problems on hypersurfaces*, Found. Comput. Math. 12 (2012), no.3, 263–293. [1](#)
- [38] Z. HU, A. JAGTAP, G. E. KARNIADAKIS AND K. KAWAGUCHI, *When do extended physics-informed neural networks (XPINNs) improve generalization?*, SIAM J. Sci. Comput. 44 (2022), no. 5, A3158–A3182. [3](#)
- [39] T. JANKUHN, M. OLSHANSKII, A. REUSKEN, AND A. ZHILIAKOV, *Error analysis of higher order trace finite element methods for the surface Stokes equation*, Journal of Numerical Mathematics, 29 (2021), pp. 245–267. [1](#)
- [40] M. JIN, X. FENG, AND K. WANG, *Gradient recovery-based adaptive stabilized mixed fem for the convection–diffusion–reaction equation on surfaces*, Computer Methods in Applied Mechanics and Engineering, 380 (2021), p. 113798. [1](#)
- [41] J. JOST, *Riemannian Geometry and Geometric Analysis*, Seventh edition, Universitext, Springer, Cham, 2017. [1](#)
- [42] D. KINGMA AND J. BA, *Adam: A Method for Stochastic Optimization*, Proceedings of the 3rd International Conference on Learning Representations, 2015. [12](#)
- [43] A. KOPANIČÁKOVÁ, H. KOTHARI, G. E. KARNIADAKIS AND R. KRAUSE, *Enhancing training of physics-informed neural networks using domain decomposition-based preconditioning*

- strategies*, SIAM J. Sci. Comput. 46 (2024), no. 5, S46–S67. 3
- [44] I. LAGARIS, A. LIKAS AND D. FOTIADIS, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Transactions on Neural Networks, 9 (1998), 987–1000. 3
- [45] Y. LEE, A. KOPANIČÁKOVÁ AND G. E. KARNIADAKIS, *Two-level overlapping additive Schwarz preconditioner for training scientific machine learning applications*, Comput. Methods Appl. Mech. Engrg. 448 (2026), part A, Paper No. 118400. 3
- [46] M. LESHNO, V. LIN, A. PINKUS AND S. SCHOCKEN, *Multilayer feedforward networks with a non-polynomial activation function can approximate any function*, Neural Networks, 6 (1993), 861–867. 3
- [47] P. L. LIONS, *On the Schwarz alternating method I*, in the First International Symposium on Domain Decomposition Methods for Partial Differential Equations, 1988. 2, 3, 4, 5, 6
- [48] P. L. LIONS, *On the Schwarz alternating method II, Stochastic interpretation and order properties*, in the Second International Symposium on Domain Decomposition Methods for Partial Differential Equations, 1989. 2, 3, 4, 5
- [49] P. L. LIONS, *On the Schwarz alternating method III, A variant for nonoverlapping subdomains*, in the Third International Symposium on Domain Decomposition Methods for Partial Differential Equations, 1990. 2
- [50] Q. LI AND J. CHEN, *An unsupervised network architecture search method for solving partial differential equations*, J. Comput. Phys. 560 (2026), Paper No. 114954. 3
- [51] Z. LI, N. KOVACHKI, K. AZIZZADENESHELI, B. LIU, K. BHATTACHARYA, A. STUART AND A. ANANDKUMAR, *Fourier Neural Operator for Parametric Partial Differential Equations*, International Conference on Learning Representations 2021, <https://openreview.net/forum?id=c8P9NQVtmnO>. 3
- [52] S. LI, Y. XIA, Y. LIU AND Q. LIAO, *A deep domain decomposition method based on Fourier features*, J. Comput. Appl. Math. 423 (2023), Paper No. 114963. 3
- [53] W. LI, X. XIANG AND Y. XU, *Deep domain decomposition method: elliptic problems*, Proceedings of The First Mathematical and Scientific Machine Learning Conference, PMLR 107:269–286, 2020, <https://proceedings.mlr.press/v107/li20a>. 3
- [54] Z. LONG, Y. LU, X. MA AND B. DONG, *PDE-Net: learning PDEs from data*, Proceedings of the 35th International Conference on Machine Learning, 3208–3216, 2018. 3
- [55] L. LYU, Z. ZHANG, M. CHEN AND J. CHEN, *MIM: a deep mixed residual method for solving high-order partial differential equations*, J. Comput. Phys. 452 (2022), Paper No. 110930. 3
- [56] J. NEDELEC, *Curved finite element methods for the solution of singular integral equations on surfaces in R^3* , Computer Methods in Applied Mechanics and Engineering, 8 (1976), pp. 61–80. 1
- [57] A. MOHAMED AND C. DAVATZIKOS, *Finite element modeling of brain tumor mass-effect from 3d medical images*, in International Conference on Medical Image Computing and Computer-assisted Intervention, Springer, 2005, pp. 400–408. 1
- [58] J. NEDELEC AND J. PLANCHARD, *Une méthode variationnelle d'éléments finis pour la résolution numérique d'un problème extérieur dans R^3* , Revue française d'automatique, informatique, recherche opérationnelle. Analyse numérique, 7 (1973), pp. 105–129. 1
- [59] P. PANG, L. LU AND G. E. KARNIADAKIS, *fPINNs: fractional physics-informed neural networks*, SIAM J. Sci. Comput. 41 (2019), no. 4, A2603–A2626. 3
- [60] D. PSICHOGIOS AND L. UNGAR, *A hybrid neural network-first principles approach to process modeling*, AIChE J. 38 (1992), 1499–1511. 3
- [61] E. QIAN, B. KRAMER, B. PEHERSTORFER AND K. WILLCOX, *Lift & learn: physics-informed machine learning for large-scale nonlinear dynamical systems*, Phys. D 406 (2020), 132401. 3
- [62] L. QIN, F. WANG AND Y. WANG, *A parallel domain decomposition method for solving elliptic equations on manifolds*, SIAM J. Sci. Comput. 47 (2025), no. 2, A916–A942, arXiv:2402.00364. 1, 2, 3, 4, 5, 7, 10
- [63] L. QIN, S. ZHANG AND Z. ZHANG, *Finite element formulation in flat coordinate spaces to solve elliptic problems on general closed Riemannian manifolds*, SIAM J. Sci. Comput. 36 (2014), no. 5, A2149–A2165. 2, 10
- [64] A. QUARTERONI AND A. VALLI, *Domain Decomposition Methods for Partial Differential Equations*, The Clarendon Press, Oxford University Press, New York, 1999. 2
- [65] M. RAISSI, P. PERDIKARIS AND G. E. KARNIADAKIS, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, J. Comput. Phys. 378 (2019), 686–707. 2, 3, 11
- [66] M. REUTER, S. BIASOTTI, D. GIORGI, G. PATANÈ, AND M. SPAGNUOLO, *Discrete Laplace–*

- Beltrami operators for shape analysis and segmentation*, Computers & Graphics, 33 (2009), pp. 381–390. [1](#)
- [67] R. SCHOEN AND S. T. YAU, *Lectures on Differential Geometry*, International Press, Cambridge, MA, 1994. [1](#)
- [68] H. A. SCHWARZ, *Ueber einige Abbildungsaufgaben*, Journal für die Reine und Angewandte Mathematik, 70 (1869), 105–120. [2](#), [5](#)
- [69] Y. SHANG, A. HEINLEIN, S. MISHRA AND F. WANG, *Overlapping Schwarz preconditioners for randomized neural networks with domain decomposition*, Comput. Methods Appl. Mech. Engrg. 442 (2025), Paper No. 118011. [3](#)
- [70] B. SMITH, P. BJØRSTAD AND W. GROPP, *Domain Decomposition, Parallel Multilevel Methods for Elliptic Partial Differential Equations*, Cambridge University Press, Cambridge, 1996. [2](#)
- [71] Q. SUN, X. XU AND H. YI, *Domain decomposition learning methods for solving elliptic problems*, SIAM J. Sci. Comput. 46 (2024), no. 4, A2445–A2474. [3](#)
- [72] Q. SUN, X. XU AND H. YI, *Dirichlet-Neumann learning algorithm for solving elliptic interface problems*, Commun. Comput. Phys. 38 (2025), no. 1, 248–284. [3](#)
- [73] J. TAYLOR, M. BASTIDAS, V. CALO AND D. PARDO, *Adaptive deep Fourier residual method via overlapping domain decomposition*, Comput. Methods Appl. Mech. Engrg. 427 (2024), Paper No. 116997. [3](#)
- [74] R. TIAN, P. KOU, Y. ZHANG, M. MEI, Z. ZHANG AND D. LIANG, *Residual-connected physics-informed neural network for anti-noise wind field reconstruction*, Applied Energy, 357 (2024), 122439. [18](#)
- [75] A. TOSELLI AND O. WIDLUND, *Domain Decomposition Methods-Algorithms and Theory*, vol. 34, Springer Science & Business Media, 2004. [2](#)
- [76] Y. WANG, Z. LIN, Y. LIAO, H. LIU AND H. XIE, *Solving high-dimensional partial differential equations using tensor neural network and a posteriori error estimators*, J. Sci. Comput. 101 (2024), no. 3, Paper No. 67. [3](#)
- [77] J. XU, *Iterative methods by space decomposition and subspace correction*, SIAM Rev. 34 (1992), no.4, 581–613. [2](#)
- [78] J. XU, *Finite neuron method and convergence analysis*, Commun. Comput. Phys. 28 (2020), no. 5, 1707–1745. [3](#)
- [79] J. XU AND J. ZOU, *Some nonoverlapping domain decomposition methods*, SIAM Rev. 40 (1998), no.4, 857–914. [2](#)
- [80] Y. ZANG, G. BAO, X. YE AND H. ZHOU, *Weak adversarial networks for high-dimensional partial differential equations*, J. Comput. Phys. 411 (2020), 109409. [3](#)